

5 Papers Across 5 Technical Domains (Cloud Digital Twins, eBPF Observability, Cascading Failure Mitigation, Federated Privacy-Preserving Monitoring, Chaos Engineering)

Paper 1 – Digital Twins for Cloud Infrastructure & Network Management

Title: 6G-Enabled Digital Twin Framework for Real-Time Cyber-Physical Systems: An Experimental Validation with Industrial Bearing Fault Detection **Authors:** Vaskar Chakma, Wooyeol Choi **Year:** 2025 (arXiv, Oct 2025) **Publisher/Venue:** arXiv preprint (Intelligent Networking Lab, Chung-Ang University, Seoul) **Link:** <https://arxiv.org/pdf/2510.03807>

Problem Statement: Existing 5G-enabled Digital Twin (DT) systems for cyber-physical systems (CPS) top out around 10ms end-to-end latency – too slow for mission-critical industrial control that needs sub-millisecond response (e.g., preventing catastrophic bearing failure in manufacturing).

Research Objective: Build and experimentally validate a 6G-enabled DT framework that achieves ultra-low-latency, real-time synchronization between a physical asset and its digital twin, using industrial bearing fault detection as the proving ground.

Methodology / Architecture: A five-layer architecture – Physical Layer (accelerometers, temp/acoustic sensors at 12kHz sampling) → Data Acquisition Layer (noise filtering, windowing, feature extraction) → 6G Communication Layer (terahertz 0.1–1 THz links, intelligent reflecting surfaces, network slicing) → Edge Intelligence Layer (Random Forest classifiers, federated learning across edge nodes) → Digital Twin Core (physics-based bearing models, Kalman-filter-based state synchronization).

Algorithms: Adaptive Digital Twin Synchronization (state prediction + Kalman filter update + error-threshold emergency re-sync trigger); 6G-Enabled Fault Detection (time+frequency feature extraction → 6G transmission → Random Forest classification).

Models Used: Random Forest (primary classifier), SVM (baseline comparator).

Datasets: Case Western Reserve University (CWRU) Bearing Data Center dataset – 4 classes (normal, inner race, outer race, ball fault), EDM-induced faults from 0.007"–0.040", multiple load conditions.

Evaluation Metrics: Accuracy, Precision, Recall, F1 (macro-averaged), end-to-end latency, throughput, ROC/AUC.

Experimental Setup: CWRU dataset, 2048-sample windows w/ 50% overlap, 15 time+frequency-domain features, compared against WiFi-6 and 5G baselines.

Cloud/Platform Context: Edge-native AI + 6G wireless – not Kubernetes-based, but directly generalizable to cloud/edge infrastructure digital twins (explicitly cited by authors as future work: power systems, transportation infrastructure).

Key Results: 97.7% classification accuracy (Random Forest) vs. 91.3% (SVM);

0.8ms end-to-end latency – a 15.6× improvement over WiFi-6 (12.5ms) and 5.25× over 5G (4.2ms); F1 > 97% across all four fault classes; AUC > 0.94 for all classes; sustained throughput up to 2000 Mbps vs. 500 Mbps traditional; sub-linear latency growth with device count (good scalability).

Strengths: Rigorous latency decomposition (sensing/comm/edge/sync/control breakdown); real dataset with real fault mechanics; strong quantitative comparison against realistic baselines (not strawmen); mathematically formalized state-

synchronization objective (Eq. 3–4) directly usable for cloud digital-twin sync-error metrics.

Weaknesses / Limitations (author-acknowledged): 6G terahertz infrastructure is not commercially deployed – the "6G Communication Layer" is largely simulated/projected, not field-tested; single asset type (bearings) tested – generalizability to other infrastructure (power, network gear) unproven; no discussion of adversarial/security robustness of the twin sync channel; Kalman-filter-based sync assumes near-linear state dynamics, which may not hold for more complex cloud infrastructure states.

Future Work Suggested by Authors: Physics-based DT models for other domains; blockchain integration for tamper-proof maintenance records; expansion to power systems and transportation infrastructure.

Open Problems / Relevance to Cloud Digital Twins: The mathematical synchronization framework (state vectors $\Phi(t)/\Psi(t)$, sync error $\varepsilon(t)$, latency decomposition) is directly portable to a cloud infrastructure digital twin – replacing "bearing vibration state" with "node/pod/network state" and "6G link" with "cloud control-plane channel." This is the strongest transferable contribution for the group's Topic 1 research.

Industrial/Government/Military Relevance: High – predictive maintenance is directly applicable to critical infrastructure (power grids, transportation, defense logistics).

Novelty Score: 8/10 (novel application of 6G+DT+edge-AI convergence; underlying components are individually known) | **Complexity:** High | **Scalability:** Demonstrated sub-linear growth, but only simulated at modest device counts | **Reproducibility:** Moderate – CWRU dataset is public, but 6G/THz layer cannot be reproduced without specialized hardware.

Paper 2 – eBPF Kernel-Level Observability (Sidecar-less Distributed Tracing)

Title: ZeroTracer: In-Band eBPF-Based Trace Generator With Zero Instrumentation for Microservice Systems **Authors:** Wanqi Yang, Pengfei Chen, Kai Liu, Huxing Zhang **Year:** 2025 **Publisher/Venue:** IEEE Transactions on Parallel and Distributed Systems (TPDS), Vol. 36, No. 7, pp. 1478–1494 **DOI:** 10.1109/TPDS.2025.3571934 **Code:** <https://github.com/IntelligentDDS/zerotracer>

Problem Statement: Existing distributed tracing solutions (OpenTelemetry + sidecar proxies) require code instrumentation or sidecar injection, both of which introduce trace loss, inaccuracy, and deployment friction in microservice systems.

Research Objective: Build an in-kernel, eBPF-based tracing system that reconstructs accurate end-to-end HTTP request traces **without any application code modification and without sidecars.**

Methodology / Architecture: eBPF programs attached to socket APIs and TCP events capture request/response boundaries in-kernel; an in-band trace generator reconstructs causality (which downstream call belongs to which incoming request) purely from kernel-observed timing and connection metadata, avoiding the need for header propagation (e.g., no reliance on X-Request-ID injection via sidecars).

Algorithms: In-band causality reconstruction from socket/TCP event correlation (as opposed to out-of-band header-based context propagation used by Envoy/Istio-style meshes).

Datasets/Evaluation: Real microservice workloads under varying concurrency levels

(not a single public benchmark — evaluated via workload concurrency stress tests).

Evaluation Metrics: Trace accuracy, performance stability under concurrency.

Key Results: >91% trace accuracy; stable performance across workload concurrency levels; outperforms other non-invasive (sidecar-less) approaches that fail to reconcile accurate request causality.

Strengths: Truly zero-instrumentation (no app code changes, no sidecar), which is the core promise of the "sidecar-less observability" research thrust; open-source implementation (GitHub) enables reproducibility; targets the specific, well-known industry pain point of sidecar overhead (CPU/memory/latency tax documented extensively — e.g., "Dissecting Overheads of Service Mesh Sidecars").

Weaknesses / Limitations: Tailored specifically to HTTP — does not claim protocol-general tracing (e.g., gRPC, raw TCP protocols would need separate handling); 91% accuracy, while strong for a non-invasive method, is still below what header-based in-band context propagation (Istio/Envoy with full instrumentation) can achieve when correctly configured; single-cluster scope — no stated multi-cluster or multi-cloud federation.

Industrial Relevance: High — directly reduces operational cost/overhead for large microservice fleets (sidecar elimination is a major KubeCon/CNCF theme, per Cilium/Hubble ecosystem growth cited in the broader literature).

Novelty Score: 8/10 | **Complexity:** High (kernel-level eBPF development, in-band causality inference is a non-trivial CS problem) | **Scalability:** Demonstrated at microservice scale, cluster-level | **Reproducibility:** High — code publicly available on GitHub.

Relevant comparator work found in the same search (for the group's broader literature map, not separately deep-dived): CrossTrace (cross-thread/cross-service span correlation, arXiv 2508.11342) and Traceweaver (ACM SIGCOMM 2024) — both tackle the same "trace without app modification" problem from different angles (span correlation vs. kernel event correlation), useful for a comparison table.

Paper 3 — Cascading Failure Mitigation & Adaptive Circuit-Breaking

Title: Multi-Objective Adaptive Rate Limiting in Microservices Using Deep Reinforcement Learning **Authors:** Ning Lyu (Carnegie Mellon Univ.), Yuxi Wang (Hofstra Univ.), Ziyu Cheng (USC), Qingyuan Zhang (Boston Univ.), Feng Chen (Northeastern Univ.) **Year:** 2025 **Publisher/Venue:** arXiv preprint **Link:** <https://arxiv.org/pdf/2511.03279>

Problem Statement: Traditional rate-limiting algorithms (token bucket, leaky bucket, fixed thresholds) require manual tuning and cannot adapt to dynamic traffic bursts or shifting system load — a major cause of cascading failure when static thresholds are wrong for current conditions.

Research Objective: Formalize adaptive rate limiting as a Markov Decision Process and solve it with a hybrid Deep Reinforcement Learning architecture that dynamically balances throughput, latency, and stability.

Methodology / Architecture: Rate limiting modeled as MDP $M=(S,A,P,R,\gamma)$ with an 8-dimensional state vector (request rate, CPU%, memory%, threshold, response time, queue length, error rate, temporal/periodicity features); discrete 7-action space (threshold adjustments from -50% to +50%); multi-objective reward combining throughput, latency, and threshold-stability terms.

Algorithms: Hybrid **DQN + A3C** architecture — DQN (3 hidden layers, 128-256-128

neurons, experience replay buffer of 100K, target network) provides stable value estimation; A3C (16 asynchronous workers, n-step advantage estimation, entropy regularization) provides efficient policy-space exploration; a dynamically-weighted fusion coefficient (α : 0.3→0.7) blends the two, shifting from exploration to exploitation over training.

Models Used: DQN, A3C (hybrid fusion), compared against Fixed Threshold, CPU-based dynamic limiting, AIMD, PID controller, and plain DQN baselines.

Datasets/Experimental Setup: DeathStarBench benchmark (Social Network + Media Service, 30+ interconnected microservices) on a 10-node AWS EC2 Kubernetes v1.28 cluster; Locust-generated periodic/burst/mixed traffic patterns.

Evaluation Metrics: Throughput (req/s), P50/P90/P99 latency, availability, resource utilization, SLA compliance rate.

Key Results: 30.9% throughput improvement and 38.2% P99 latency reduction vs. fixed-threshold baseline; 98.7% SLA compliance (500ms threshold); 2–5ms decision latency (real-time capable); **90-day production deployment at 500M requests/day** showed 82% reduction in service degradation incidents and 68% decrease in manual interventions; ablation shows experience replay is the single most critical component (9.7% degradation if removed).

Strengths: Rare combination of rigorous MDP formalization and real 90-day production validation at genuine scale (500M req/day) – most academic RL-for-ops papers stop at simulation; strong ablation study isolating each architectural component's contribution; directly implementable on Kubernetes/Envoy (uses Envoy sidecar metrics, Prometheus, Runtime Discovery Service for live config push).

Weaknesses / Limitations (author-acknowledged): ~48 hours of GPU training required before deployment – a real cold-start cost; black-box interpretability problem for SRE/ops teams (authors suggest attention mechanisms or decision-tree extraction as future mitigation); single-service rate limiting – does **not** yet coordinate rate limits across interdependent services, which is precisely the cascading-failure scenario (a locally-optimal rate limit at Service A can still starve Service B).

Future Work Suggested by Authors: Multi-agent coordination across interconnected microservices (global rather than local optimization) – this is the identified gap most relevant to "cascading failure mitigation" specifically; meta-learning/transfer learning for cold-start; model compression for edge deployment; explainable AI (SHAP, attention) for trust.

Industrial Relevance: Very high – validated at hyperscale production traffic;

Government/Military Relevance: Applicable to any mission-critical distributed system needing graceful degradation under load (e.g., emergency services APIs, banking transaction gateways).

Novelty Score: 7/10 (DRL for rate limiting is an active area; the hybrid DQN-A3C fusion mechanism and the production-scale validation are the differentiators) |

Complexity: High | **Scalability:** Proven at 500M req/day | **Reproducibility:** Moderate – architecture and hyperparameters are fully specified (Table 1), but no code repository mentioned.

Paper 4 – Federated & Privacy-Preserving Cloud Monitoring

Title: FedMon: Federated eBPF Monitoring for Distributed Anomaly Detection in Multi-Cluster Cloud Environments **Authors:** Sehar Zehra, Hassan Jamil Syed,

Ummay Faseeha **Year:** 2025 (arXiv, Oct 2025) **Publisher/Venue:** arXiv preprint (FAST National University, Karachi / Asia Pacific University, Kuala Lumpur) **Link:** <https://arxiv.org/pdf/2510.10126>

Problem Statement: eBPF-based monitors (Falco, Cilium, KubeArmor) give excellent low-overhead visibility but are confined to a single Kubernetes cluster; centralized cross-cluster monitoring solves the visibility gap but incurs bandwidth overhead and **privacy risk from shipping raw telemetry** across cluster/tenant boundaries – directly the "PNR/PII" concern named in the assignment brief.

Research Objective: Unify kernel-level eBPF telemetry with Federated Learning so multiple Kubernetes clusters can collaboratively learn an anomaly-detection model **without any raw syscall/network data leaving its cluster of origin.**

Methodology / Architecture: Per-cluster eBPF DaemonSet agents attach to syscall/network tracepoints → local feature pipeline converts raw events into compact features (frequency histograms, sequence embeddings) so **raw data never leaves the cluster** → each cluster trains a local Variational Autoencoder (VAE) and sends only encrypted, clipped model updates (not features, not raw data) to a global FL server → global server aggregates via FedAvg (default) or Krum (Byzantine-robust alternative) → aggregated model redistributed → each cluster fuses the global VAE with a local Isolation Forest for cluster-specific outlier scoring → risk-aware enforcement via eBPF/LSM hooks (logging → throttling → blocking, scaled by anomaly severity).

Algorithms: Federated VAE (global representation learning) + local Isolation Forest (cluster-specific fine-tuning); FedAvg / Krum aggregation for Byzantine robustness.

Datasets/Experimental Setup: 3 emulated Kubernetes clusters (kind/Docker) on a single host (AMD Ryzen 9 5900X, 64GB RAM); standardized nginx/redis workloads; injected attacks: crypto-mining (CPU-heavy loops), data exfiltration (DNS/HTTP POST), reverse shell attempts.

Evaluation Metrics: F1-score, precision, recall, application throughput, inter-cluster bandwidth, robustness under simulated poisoning.

Key Results: 94% precision, 91% recall, **F1 = 0.92**; **>60% bandwidth reduction** vs. centralized raw-telemetry streaming; near-baseline application throughput (minimal overhead vs. agent-only baseline); Krum aggregation successfully preserves accuracy under a simulated poisoning attack in round 3, where standard FedAvg degrades.

Strengths: Directly and explicitly solves the "process telemetry without transmitting sensitive PII/raw data" requirement – this is the single closest match in the entire search to the assignment's Topic 4 description; combines two of the assignment's other topics (eBPF + Federated Learning) in one system, making it a strong cross-cutting reference; includes an explicit adversarial threat model (honest-but-curious server, Byzantine clients) and empirically validates robust aggregation, not just accuracy.

Weaknesses / Limitations (author-acknowledged): Evaluated only on 3 co-located (single-host, kind-based) clusters – real geo-distributed, cross-datacenter WAN latency and bandwidth effects are untested; attack set is representative but narrow (3 attack types only); differential privacy is mentioned as "optional" and "tunable" but not rigorously quantified (no epsilon/delta reported for the DP mechanism); scalability to "dozens or hundreds of clusters" explicitly left as future work.

Future Work Suggested by Authors: Hierarchical FL for WAN-scale (dozens/

hundreds of clusters); personalization methods (FedPer, FedProx) for non-IID workloads; extend beyond syscalls to L7/application logs; formal verification of aggregation + DP guarantees; TEE integration for secure local training; a Kubernetes Operator for lifecycle management.

Industrial/Government Relevance: Very high — multi-tenant cloud providers, government multi-agency cloud (where data sovereignty across departments is a hard requirement), and banking (cross-branch fraud detection without centralizing customer telemetry) are direct use cases.

Novelty Score: 9/10 — first framework to explicitly integrate eBPF kernel telemetry with FL for cross-cluster anomaly detection (per the authors' own related-work table, no prior work combines both) | **Complexity:** High | **Scalability:** Unproven beyond 3 co-located clusters — this is the paper's main limitation | **Reproducibility:** Moderate — uses named open frameworks (Flower for FL) but no public code repo cited.

Paper 5 — Chaos Engineering-Based Resilience Modeling for Critical Systems

Title: Resilience Evaluation of Kubernetes in Cloud-Edge Environments via Failure Injection **Authors:** Zihao Chen, Mohammad Goudarzi (Monash University), Adel Nadjaran Toosi (University of Melbourne) **Year:** 2025 (arXiv, Jul 2025) **Publisher/Venue:** arXiv preprint **Link:** <https://arxiv.org/pdf/2507.16109> **Code:** <https://github.com/dylanC777/cloud-edge-k8s-resilience>

Problem Statement: Systematic, quantitative resilience assessment of Kubernetes under realistic fault conditions in **hybrid cloud-edge** environments is sparse; no prior work had directly and empirically compared monolithic vs. microservice architectural resilience under identical fault conditions across the cloud-edge continuum.

Research Objective: Build an automated fault-injection + workload-generation orchestration framework and use it to run a large-scale, systematic resilience study comparing cloud-only vs. cloud-edge hybrid Kubernetes deployments, and monolithic vs. microservice application architectures.

Methodology / Architecture: Six-layer orchestration framework — Core Components (Cluster Checker for health validation, Main Module as central orchestrator) → Controller Components (Recovery Controller, Chaos Scheduler, Injection Controller integrating **Chaos Mesh** as the primary fault engine, extensible to Gremlin/ChaosBlade) → Load Generation System (**Locust**, 3 modes: piggyback, concurrent, constant-rate) → Result Manager (metric extraction, correlation analysis between fault timing and observed effects) → Kubernetes Cluster Integration Layer → object-oriented modular design (Facade/Mediator/Template Method/Singleton patterns) enabling one-click, agentless (SSH-based) experiment execution across 5 phases: health check → fault injection → load generation → data collection → recovery verification.

Algorithms/Techniques: z-score normalization of resilience metrics relative to a 25%-fault-intensity baseline, enabling cross-environment comparison despite differing absolute baselines — this is the paper's key methodological contribution (dual-metric: absolute response time + normalized volatility).

Datasets: **First comprehensive resilience dataset for hybrid cloud-edge Kubernetes** — 11,965 fault-injection experiments, 57M+ request-level records, ~30GB of structured time-series logs, across 4-node and 8-node clusters in both pure-cloud and cloud-edge-hybrid configurations. Applications: monolithic Image-Detection

(deep-learning classifier) vs. microservices Sock-Shop (13 interdependent services).

Fault Types: Container/pod termination, network delay injection, network loss, bandwidth throttling, CPU stress, node termination — each at 4 intensity levels (25/50/75/100%).

Evaluation Metrics: Mean/P95 response time, failure rate, z-score-normalized resilience/volatility.

Key Results: 8-node clusters show lower latency and failure rates than 4-node (expected, but now quantified); **microservices show substantially higher tail latency and failure rates than monolithic** even controlling for scale/faults — 43.4% failure rate for Sock-Shop vs. 18.8% for Image-Detection on the 4-node cluster under identical fault conditions; **cloud-edge hybrid deployments show 80% superior response stability under network delay/partition** (fewer hops = less cascading delay amplification); **cloud-only deployments show 47% better resilience under bandwidth limitation and network loss** (better redundancy/retry paths); reveals a genuine "resilience inversion" — edge is slower in absolute terms but more stable under certain fault types, challenging simple "cloud is always better" intuition.

Strengths: By far the largest and most rigorous dataset among the five papers (11,965 experiments, 57M requests) — directly usable as a benchmark/reference dataset for the group's own chaos experiments; open-source framework (GitHub) supports direct reproducibility; z-score normalization methodology is a genuinely reusable analytical technique the group could adopt for their own resilience-testing chapter; direct monolith-vs-microservice comparison under identical fault conditions fills a specific, named literature gap.

Weaknesses / Limitations (author-acknowledged): All experiments run on virtualized/simulated infrastructure (NeCTAR Research Cloud) with simulated edge conditions (artificial 200ms latency, 10% packet loss) rather than real geographically-distributed edge hardware; only 2 applications tested (1 monolith, 1 microservice app) — architectural generalizability beyond these two specific apps is unproven; does not test control-plane-level faults as deeply as the cited Mutiny framework (etcd corruption); dataset itself not yet publicly released at time of writing (available on request pending publication).

Future Work Suggested by Authors: Integrate additional fault-injection/load-generation platforms; validate in real (non-simulated) edge deployments; broader fault coverage across system layers (e.g., control-plane); improve framework modularity.

Industrial/Government/Military Relevance: High — the cloud-vs-edge resilience trade-off findings are directly actionable for telecom, industrial IoT, and any latency-sensitive critical infrastructure deciding where to place workloads.

Novelty Score: 8/10 (framework unifies control-plane + pod + network fault injection + load-gen + cloud-edge coordination in one tool — no prior tool in their own comparison table, reproduced below, covers all six dimensions) | **Complexity:**

Medium-High | **Scalability:** Demonstrated at 4- and 8-node cluster scale |

Reproducibility: High — code publicly available; dataset available on request.

Reproduced Tool Comparison (from the paper, useful for the group's own comparative table):

Tool	Pod/Node Faults	Network Faults	Control- Plane	Load Gen	App Metrics	Cloud- Edge
------	--------------------	-------------------	-------------------	-------------	----------------	----------------

Chaos
Mesh
Gremlin
ChaosBlade
Mutiny
**This
work**

Cross-Paper Summary Table

Topic	Paper	Venue/ Year	Key Metric	Novelty	Reproducibility
Digital Twins	6G-Enabled DT Framework	arXiv 2025	0.8ms latency, 97.7% acc.	8/10	Moderate
eBPF Observability	ZeroTracer	IEEE TPDS 2025	91%+ trace accuracy	8/10	High (code public)
Cascading Failure Mitigation	Multi-Objective Adaptive Rate Limiting (DQN-A3C)	arXiv 2025	30.9% throughput ↑, 38.2% P99 ↓	7/10	Moderate
Federated Privacy-Preserving Monitoring	FedMon	arXiv 2025	F1=0.92, 60% + bandwidth ↓	9/10	Moderate
Chaos Engineering	Cloud-Edge K8s Resilience via Failure Injection	arXiv 2025	11,965 experiments, 80%/47% stability gains	8/10	High (code public)

Notable Cross-Cutting Gap Observed

Papers 2 and 4 both use eBPF as their observability substrate but solve different problems (tracing vs. federated anomaly detection) — no paper in this set combines **sidecar-less eBPF tracing** with **federated cross-cluster learning** and **digital-twin state synchronization** in one system. That three-way integration gap (Topics 1+2+4 combined) is a candidate direction worth flagging to the rest of the team for the "innovation discovery" phase of the larger project.